

INGENIERÍA DE DATOS II

Tecnicatura / Licenciatura / Ingeniería en Informática

TRABAJO PRÁCTICO INTEGRADOR 2.ª ENTREGA

TEMA 12

Municipio Digital — Gestión de Trámites Ciudadanos

Motores — 1.a Entrega	MongoDB · Neo4j
Motor adicional	Cassandra
Capa poliglota	MongoDB + Neo4j + Cassandra
Unidades evaluadas	V — Acceso desde aplicaciones · Persistencia Poliglota
Fecha de entrega	Lunes 1 de junio de 2026
Modalidad	Grupal (5 integrantes) — extensión de la 1.a Entrega
Defensa oral	Luego de corregidas ambas entregas (fecha a confirmar)

1. Introducción

Este documento extiende la 1.a Entrega

Esta segunda entrega no reemplaza el trabajo ya entregado en la primera instancia. Lo extiende. El grupo debe presentar este documento junto con el código adicional que implementa el tercer motor y la capa de persistencia poliglota. Todo lo desarrollado en la 1.a Entrega sigue vigente y forma parte del sistema completo que se defenderá oralmente.

La primera entrega estableció el núcleo: los trámites ciudadanos y su historial de eventos en MongoDB, y el flujo de derivaciones entre organismos en Neo4j. Esta segunda entrega incorpora dos nuevos desafíos:

- **Tercer motor — Cassandra:** incorporación de Cassandra para gestionar el log de eventos de trámites a gran escala (3.000 solicitudes diarias × múltiples eventos cada una) y las métricas de desempeño de cada organismo en tiempo real.
- **Capa de persistencia poliglota:** implementación de una aplicación con interfaz mínima que integra los tres motores en operaciones de negocio cohesivas, decidiendo conscientemente qué datos consulta en cada motor y cómo ensambla la respuesta.

Cada decisión de diseño debe estar justificada en el informe escrito.

2. Tercer Motor: Cassandra

2.1 Justificación de incorporación

El dominio municipal tiene tres naturalezas de datos claramente diferenciadas:

- **Datos históricos y estructurados:** los trámites con su estructura variable por tipo y los datos maestros de ciudadanos y organismos viven en MongoDB.
- **Datos relacionales y de red:** el flujo de derivaciones entre organismos y las dependencias entre trámites viven en Neo4j.
- **Datos de alta escritura y volumen masivo:** el log de eventos de trámites (cada trámite genera entre 5 y 15 eventos a lo largo de su ciclo de vida) y las métricas de desempeño de cada organismo (tiempo promedio de respuesta, trámites procesados por hora) son datos de alta frecuencia de escritura que Cassandra gestiona de forma óptima por organismo y período.

Cassandra resuelve el volumen del log de eventos con particionamiento por organismo y fecha, permitiendo análisis de desempeño en tiempo real sin afectar el sistema transaccional de MongoDB.

Error conceptual frecuente — Cassandra no es un reemplazo de MongoDB

Cassandra no debe usarse para consultas analíticas complejas ni para datos con relaciones variables. Su fortaleza es la escritura masiva y la lectura por clave de partición con altísimo volumen. MongoDB sigue siendo la fuente de verdad estructurada. Cassandra gestiona el volumen de eventos.

2.2 Modelado en Cassandra — Modelo de tablas

El grupo debe diseñar las tablas Cassandra para el log de eventos y las métricas de organismos:

Tabla Cassandra	Caso de uso en el dominio	Justificación técnica
eventos_tramite (partition: organismo_id + fecha, clustering: timestamp DESC)	Log de todos los eventos procesados por un organismo en una fecha: tramite_id, tipo, duración, agente	Permite analizar el flujo de trabajo de un organismo en un día sin scan global.
eventos_por_tramite (partition: tramite_id, clustering: timestamp ASC)	Todos los eventos de un trámite específico en orden cronológico: tipo, organismo, agente, detalle	Permite reconstruir el historial completo de un trámite en O(1) por partición.
metricas_organismo (partition: organismo_id + anio_mes, clustering: fecha)	Métricas diarias de un organismo: trámites procesados, tiempo promedio, SLA cumplidos/incumplidos	Permite analizar la evolución del desempeño de un organismo mes a mes.
demanda_tipo_tramite (partition: tipo_tramite + anio_mes, clustering: fecha)	Demanda mensual por tipo de trámite: total solicitudes, distribución por zona de la ciudad	Permite identificar qué tipos de trámite crecen más y desde qué zonas.
notificaciones_ciudadano (partition: ciudadano_id, clustering: timestamp DESC)	Notificaciones enviadas a un ciudadano: tipo, tramite, canal, estado de lectura	Permite mostrar el historial de comunicaciones de un ciudadano sin scan global.

Diseño de partition key y clustering key

Principio clave: cada tabla Cassandra responde a una query específica.

En este dominio se necesitan dos vistas del mismo evento:

- Por organismo: para análisis de desempeño del organismo (quién procesó qué y cuándo)
- Por trámite: para reconstruir el historial de un ciudadano (qué pasó con mi trámite)

Esto implica escribir el mismo evento en dos tablas diferentes. Es una duplicación intencional.

Ejemplos de partition keys:

eventos_tramite: partition=(organismo_id, fecha) → flujo de trabajo del organismo en un día

eventos_por_tramite: partition=tramite_id → historial completo del trámite

metricas: partition=(organismo_id, anio_mes) → evolución mensual del organismo

Justificar en el informe la duplicación y qué trade-off implica.

3. Requerimientos de Implementación — Cassandra

El grupo deberá implementar y documentar las siguientes operaciones en Cassandra:

3.1 Log de eventos de trámites

1. Implementar el registro de eventos de trámites en Cassandra. Cada evento debe incluir: organismo_id, tramite_id, tipo (recepcion/derivacion/resolucion/observacion), agente_id, timestamp, duración_en_organismo, detalle.
2. Implementar las siguientes consultas:
 - a) Obtener todos los eventos procesados por un organismo en una fecha dada.
 - b) Reconstruir el historial completo de un trámite: todos sus eventos en orden cronológico.
 - c) Identificar los organismos que procesaron más eventos en la última hora.
 - d) Calcular el tiempo promedio que los trámites pasan en un organismo determinado.

3.2 Métricas de desempeño por organismo

3. Implementar el registro y consulta de métricas diarias y mensuales de cada organismo.
4. Implementar las siguientes operaciones:
 - a) Actualizar las métricas del día al procesar un evento (trámites procesados, tiempo promedio, SLA cumplido).
 - b) Obtener el reporte mensual de desempeño de un organismo.
 - c) Comparar el desempeño de dos organismos en el mismo período.
 - d) Detectar organismos con tasa de incumplimiento de SLA superior al 20% en el último mes.

3.3 Análisis de demanda ciudadana

5. Implementar consultas de análisis de demanda por tipo de trámite y zona geográfica.
6. Implementar las siguientes consultas:
 - a) Obtener la demanda mensual de un tipo de trámite en el último año.
 - b) Identificar los tipos de trámite con mayor crecimiento de demanda interanual.
 - c) Detectar zonas de la ciudad con mayor concentración de un tipo de trámite específico.
 - d) Justificar por qué estas consultas de demanda viven en Cassandra y no en MongoDB.

4. Capa de Persistencia Poliglota

¿Qué es la persistencia poliglota?

La persistencia poliglota es una decisión arquitectural: distintos motores de base de datos gestionan distintas partes del dominio según sus fortalezas. No es simplemente 'usar tres bases de datos'. Implica diseñar explícitamente qué datos viven en cada motor, cómo fluyen entre ellos, cómo se mantiene la coherencia y qué motor responde cada tipo de consulta. (Harrison, 2015, Cap. 1; Pivert, 2018, Cap. 2)

4.1 Responsabilidades por motor

El grupo debe documentar explícitamente qué responsabilidad tiene cada motor en el sistema:

Motor	Responsabilidad principal	Datos que gestiona
MongoDB	Fuente de verdad histórica	Trámites con estructura variable por tipo, datos maestros de ciudadanos y organismos.
Neo4j	Red de relaciones y grafos	Flujo de derivaciones entre organismos, dependencias entre trámites, cuellos de botella.
Cassandra	Series temporales y alta escritura	Log de eventos a gran escala, métricas de desempeño por organismo, análisis de demanda.

Atención — coherencia entre motores

Cuando ocurre un evento de negocio relevante, deben actualizarse los motores que correspondan. El grupo debe documentar en el informe cómo gestiona esta coherencia y qué sucede si una de las escrituras falla (estrategia de manejo de errores parciales).

Ejemplo: cuando un organismo procesa y deriva un trámite:

- Cassandra: registra el evento en dos tablas (por organismo y por trámite); actualiza las métricas del organismo.
- MongoDB: actualiza el estado actual del trámite y el organismo que lo tiene ahora.
- Neo4j: registra la derivación real como arista en el grafo de flujo (para análisis de cuellos de botella).

4.2 Operaciones poliglotas requeridas

La aplicación debe implementar exactamente 5 operaciones que integren múltiples motores. Las operaciones marcadas con (*) deben usar los tres motores simultáneamente en una sola respuesta.

Operación de negocio	MongoDB	Neo4j	Cassandra
OP-1 Panel de seguimiento para el ciudadano (*) — 3 motores	Datos del trámite: tipo, fecha de inicio, organismo actual, documentos adjuntos	Cuántas etapas faltan hasta la resolución final según el flujo del tipo de trámite	Historial completo de eventos del trámite en orden cronológico
OP-2 Procesamiento de un evento de trámite (*) — 3 motores	Actualiza el estado actual del trámite y el organismo responsable	Registra la derivación real como arista en el grafo de flujo	Registra el evento en las dos tablas Cassandra (por organismo y por trámite); actualiza métricas
OP-3 Panel de desempeño de un organismo — 2 motores	Lista de trámites activos bajo su responsabilidad con SLA en riesgo	—	Métricas del día: trámites procesados, tiempo promedio, SLA cumplidos; evolución de la última semana
OP-4 Detección de cuellos de botella en el flujo — 2 motores	—	Organismos con mayor centralidad en el grafo (por donde pasan más tipos de trámite)	Organismos con tiempo promedio de procesamiento superior al SLA en el último mes
OP-5 Reporte ejecutivo mensual (*) — 3 motores	Total de trámites por tipo, tiempos de resolución promedio, nuevos ciudadanos registrados	Tipos de trámite con mayor complejidad de flujo (más derivaciones), organismos más críticos	Métricas de desempeño de cada organismo, evolución de la demanda por tipo de trámite

A continuación, se detalla el comportamiento esperado de cada operación:

OP-1 Panel de seguimiento para el ciudadano (*) — 3 motores

El ciudadano consulta el estado de su trámite y recibe una vista completa:

- Cassandra: historial completo del trámite (todos sus eventos en orden cronológico) desde eventos_por_tramite.
- MongoDB: datos actuales del trámite: tipo, organismo que lo tiene ahora, documentos requeridos pendientes.
- Neo4j: posición actual en el flujo de derivaciones y estimación de etapas restantes.

El informe debe documentar el ensamblado de la vista del ciudadano y el tiempo de respuesta esperado.

OP-2 Procesamiento de un evento de trámite (*) — 3 motores

Cuando un agente registra un evento en un trámite (derivación, resolución, observación), el sistema actualiza los tres motores:

- Cassandra: INSERT en eventos_tramite (para métricas del organismo) e INSERT en eventos_por_tramite (para historial del ciudadano).
- MongoDB: actualiza el campo 'organismo_actual' y 'estado' del trámite; registra el evento en el embedded array del historial.
- Neo4j: crea la arista de derivación real entre los organismos involucrados.

El informe debe justificar por qué el evento se escribe en dos tablas Cassandra y qué trade-off implica.

OP-3 Panel de desempeño de un organismo — 2 motores

El responsable de un organismo ve el estado operativo de su área:

- Cassandra: métricas del día actual y evolución de los últimos 7 días para el organismo.
- MongoDB: lista de trámites activos en el organismo con fecha de ingreso y SLA de vencimiento próximo.

El informe debe justificar por qué Neo4j no participa en este panel.

OP-4 Detección de cuellos de botella en el flujo — 2 motores

El sistema identifica qué organismos generan demoras sistémicas en el flujo de trámites:

- Cassandra: ranking de organismos por tiempo promedio de procesamiento en el último mes.
- Neo4j: para los organismos más lentos, analiza su centralidad en el grafo para evaluar el impacto de la demora en el sistema completo.

El informe debe documentar cómo se combinan los datos de Cassandra y Neo4j para priorizar intervenciones.

OP-5 Reporte ejecutivo mensual (*) — 3 motores

El área de modernización genera el reporte ejecutivo mensual del municipio digital:

- Cassandra: métricas mensuales de desempeño de cada organismo; demanda por tipo de trámite.
- MongoDB: totales de trámites por estado y tipo, tiempos de resolución promedio, nuevos ciudadanos.
- Neo4j: tipos de trámite con mayor complejidad de flujo y organismos con mayor centralidad en la red.

El informe debe documentar el ensamblado del reporte y el orden de consultas para minimizar el tiempo de generación.

5. Interfaz de la Aplicación

La capa poliglota debe implementarse como una aplicación con interfaz mínima que permita ejecutar las 5 operaciones sin necesidad de modificar el código fuente. El grupo puede elegir entre:

- **CLI (Command Line Interface):** la aplicación acepta comandos y argumentos por línea de comandos.
- **API REST:** la aplicación expone endpoints HTTP invocables desde un cliente o navegador.
- **Menú interactivo:** la aplicación presenta un menú numerado en la consola para seleccionar la operación y cargar parámetros.

Requisitos mínimos de la interfaz

Independientemente de la modalidad elegida, la interfaz debe cumplir:

1. Las 5 operaciones políglotas deben ser invocables sin modificar el código.
 2. Las conexiones a los tres motores deben configurarse mediante variables de entorno o archivo de configuración (no hardcodeadas).
 3. Los errores de conexión o de datos deben devolver mensajes descriptivos, no stack traces crudos.
 4. El README del repositorio debe incluir instrucciones claras para ejecutar la aplicación.
- Justificar en el informe la modalidad elegida y por qué es adecuada para el caso de uso.

6. Informe Escrito — Estructura Obligatoria

El informe de esta segunda entrega es un documento adicional al de la primera entrega. Debe cubrir únicamente los contenidos nuevos:

7. **Introducción:** qué agrega esta entrega al sistema de la primera entrega y cómo se articula con lo ya entregado.
8. **Justificación de Cassandra:** por qué Cassandra para este dominio, qué problema resuelve que MongoDB y Neo4j no pueden resolver, qué alternativas se descartaron.
9. **Modelado en Cassandra:** diseño de tablas, partition keys, clustering keys y justificación de cada decisión de modelado.
10. **Diseño de la capa poliglota:** tabla de responsabilidades por motor, diagrama de flujo de datos para cada operación, decisiones de coherencia.
11. **Operaciones políglotas:** para cada una de las 5 operaciones: flujo de consultas, orden de motores, ensamblado de respuesta y estrategia ante fallos.
12. **Coherencia entre motores:** qué sucede ante un fallo parcial en una escritura multi-motor. Qué garantías ofrece el sistema y cuáles no.
13. **Comparación con arquitectura puramente relacional:** elegir una de las 5 operaciones y mostrar cómo se resolvería en SQL puro. Analizar diferencias en complejidad, rendimiento y mantenibilidad.
14. **Conclusiones:** reflexión crítica sobre la arquitectura poliglota: qué ganó el sistema con este diseño y qué complejidad adicional introdujo.
15. **Bibliografía:** incorporar la documentación oficial de Cassandra y la bibliografía complementaria correspondiente.

7. Bibliografía de Referencia

Obligatoria — incorporada en esta entrega

- Apache Software Foundation. (s/f). Apache Cassandra Documentation. <https://cassandra.apache.org/doc/>
- Harrison, G. (2015). Next Generation Databases: NoSQL, NewSQL, and Big Data. Apress. (Cap. 1 — Polyglot Persistence)

Complementaria

- Bradberry, R., & Lubow, E. (2013). Practical Cassandra: A Developer's Approach. Addison-Wesley.
- Pivert, O. (Ed.). (2018). NoSQL Data Models: Trends and Challenges. ISTE.

Documentación oficial — toda la entrega

- MongoDB, Inc. (s/f). MongoDB Documentation. <https://www.mongodb.com/docs/>
- Neo4j, Inc. (s/f). Neo4j Documentation. <https://neo4j.com/docs/>
- Apache Software Foundation. (s/f). Apache Cassandra Documentation. <https://cassandra.apache.org/doc/>